

★ Chapter 7

› optimization for efficient scoring n ranking

- **Optimization**: instead of using normalized query vector $v(q)$, we use simplified vector $V(q)$, where all **non-zero values(present terms) = 1**
- **Query normalization** can be **skipped** in ranking because it only scales scores by a constant and does not affect document order!! And also cuz we only want ranking and dont care about exact values
- Faster multiplication + simple + ranking remains same
- What we're doing:
 1. Looks at **query words**
 2. Finds documents containing those words
 3. **Adds up scores** for each document
 4. Returns the **top K best documents**

Cosine similarity becomes: 👉 **Sum of term weights in the document**

› query weights =1 :

- So instead of: $\text{score} += w_{t,q} \times w_{t,d}$
- We do: **score += wt,d**
- **Using inverted index:**
 - For each term in query:
 1. Go to its **postings list**
 2. **For each document:**
 - **Add** its **score** [tf or tf-idf] $w_{t,d} = tf_{t,d} \times idf_t$
 3. Find **top K**
 - **Heap(priority queue)**
 - Only compute for docs appearing in at least one of the posting lists of the query terms

- Total docs N, J contain query terms ; only J docs are scored. J is significantly smaller than N.
- **For extracting top K: Building** the heap takes about **2J comparisons**, and then **extracting** each of the top K documents takes **log J time**. [J → no of docs containing query terms]
-
- **Tf - term frequency**

$tf_{t,d}$ = number of times term t appears in document d

$$tf_{t,d} = \frac{\text{number of times term } t \text{ appears in } d}{\text{total number of terms in document } d}$$

$$tf_{t,d} = 1 + \log(\text{frequency})$$

> reduce affect of v frequent words

- **Idf - inverse doc freq** > **how rare/imp a word is**

$$idf_t = \log \left(\frac{N}{df_t} \right)$$

N = total number of documents

df_t = number of **documents containing term t**

High idf ⇒ rare term

> in exact top K doc retrieval

- **Exact Top K** → find the **absolute best K documents**
- **Inexact Top K** → find documents that are **very close to the best K**
- Trade precision for **speed** [speed more imp than perfect accuracy]
- Users cant tell tiny diff in scores
- System returns doc that are **VERY LIKELY to be among the best**
- **Goal:**
 - **Reduce** computation **cost**
 - **Avoid scoring too many documents**
 - Still **return high-quality results**

- **IDEA:**

- **Reduce N** (tot docs) **to a smaller set A** (smaller subset of J) (**contenders**)
 - $J \rightarrow$ docs containing query terms
 - $A \Rightarrow$ contains docs that r likely to be among best!
 - **$K < |A| \ll N$**
 - We compute scores only for A and use heap/sorting to return top best K
- Example heuristics to filter out docs for A : "only consider docs where query terms r frequent"
 - Inexact top k retrieval $\Rightarrow$ **Not suitable for all query types**
 - Designed for free-text queries $\rightarrow$ flexible ranking
 - Precision critical for Boolean/phrased queries so can't use w them

› index elimination

- **Elimination** of **less useful docs** or **terms** in the **inverted index** before scoring

1) **Ignore LOW - IDF terms**

- Only process terms w **high idf / exceeds a threshold**
- Ignore common words - treated like stop words
- **Adjustable threshold** $\rightarrow$ IDF threshold can change per query
- Query term has V few terms? Lower threshold to avoid losing info

2) **Require many query terms in doc**

- Only consider doc containing many/all query terms
- Intuition: docs containing more query terms r more relevant
- During posting traversal, keep docs that match several/all query terms
- **PROBLEML**
 - Might eliminate too many docs
 - Ending up w fewer than K results

› champions / fancy lists / top docs

- For each term, store only **top r documents** [highest tf / tf-idf weights]
- **Precomputed during indexing**, so at query processing we don't have to look at all docs, just these top r
- For query q
 - Take **union** of **champion lists** of each of its terms
 - Call this set A
 - Only score these docs
- r = size of champion lists ; **$r > K$**
- **Problem: r is Fixed** but **K is Dynamic**
 - r chosen at indexing time
 - K chosen at query time
 - So we either **increase r** or switch to using **full postings**
- **Different r for diff terms:**
 - **Rare terms** → **large r** [want to include more docs]
 - **Common terms** → **small r** [less useful]

› static quality scores $g(d)$ and ordering

- Static quality scores **$g(d)$ > query independent scores for each doc**
- Value between: 0 - 1
- Measure overall quality of doc
- Precomputed and reused for all searches
- **Final score = quality + relevance(cosine sim)**

$$\text{net-score}(q, d) = g(d) + \frac{V(q) \cdot V(d)}{|V(q)| |V(d)|}$$

- Doc shdnt only match query but also be of good quality

- **Ordering Postings by Quality:**

- Sort postings by $g(d)$ instead of DOCID
- Now on traversal, we **encounter high quality docs first** → increased chances of finding top results early

› **Global Champion Lists (Extension)**

- Combines
 - **Quality $g(d)$**
 - **Tf-idf weighting**
- Keep top r docs per term
- Improved champion lists
- Query processing:
 - Take **union** of **champion lists**
 - **Compute scores** only for them
 - Faster retrieval

› **Two-Level Lists: High and Low**

- For each term:
 - **High list** → **top m docs** [best documents (high tf or importance)]
 - **Low lists** → **remaining docs** [less imp docs]
- At query time:
 - First, **process only** the **high lists**
 - If we get **enough results** (k), **stop early**
 - **Otherwise**, continue with **low lists**

› Impact Ordering

When **postings are ordered** based on their **contribution to the score**, this general approach is called **impact ordering**

What Was Happening Before?

- All postings lists had a **common order** (docID or quality)
- Allowed **document-at-a-time scoring** → where we process one document at a time and compute its full score using all query terms.

what changes in impact ordering

- Postings are sorted by decreasing **term freq (tf)**
- No common ordering across lists
- Switch to **term-at-a-time scoring**
- Now we can't traverse all list tgt and switch to processing one term at a time

› Idea 1: Process Only a Prefix of Postings

- Don't process full postings list
- **Stop early:**
 - **after r documents**, OR
 - when **tf drops below threshold**

› idea 2: process query terms by IDF

- Process **high- idf terms first**
- **Contribute more** to score

Adaptive Stopping

- Check if **scores improve significantly**
- If **not** → **stop early**

- Adding another term doesn't significantly change scores? That term isn't very useful so we can **stop processing further terms** or **process only a small portion of their postings**.
- Adaptive + efficient

Focus on High Impact

- **Prioritize high TF** and **IDF**.
- Ignore low impact contributions

Although for impact ordering, basic idea uses tf

Our approach can be generalized by using tf-idf / combination of static quality score and relevance

> Cluster Pruning:

- **Group docs** into **clusters**
- At **query time** → **search only one (or few) clusters**
- based on the idea that **similar documents are close to each other** in **vector space**. → group them into clusters
- Query arrives, find **cluster closest to it** and only **search within that cluster**

clustering step

- Pick **$\sqrt{N}$ documents randomly** → **leaders** [**representatives of clusters**]
- Remaining docs = **followers assigned to closest leader** based on **cosine sim**
- Number of leaders = $\sqrt{N}$
- Followers per leader $\approx \sqrt{N}$ [**balanced clusters**]
- Why random leaders:
 - **DENSE** regions → more leaders
 - **SPARSE** regions → fewer leaders

query processing step

- **Compare query** w all leaders
- **Choose closest** leader
- set **A = leader + its followers**
- Only these docs considered

› Improved Version with Parameters (b_1, b_2)

- $b_1 \rightarrow$ **number of leaders per follower**
- $b_2 \rightarrow$ number of **leaders checked at query time**
- To **improve accuracy** $\rightarrow$ assign each follower to multiple leaders (b_1 closest leaders)
- At query time search b_2 closest leaders

› Comparison of Efficient Retrieval Techniques

Technique	Core Idea	How It Reduces Work	Key Data Structure / Trick	Advantages	Disadvantages
Index Elimination (7.1.2)	Ignore unimportant terms/docs	Skip low-idf terms and docs with few query terms	IDF threshold, term filtering	Very simple, reduces large postings lists	May remove useful documents; risk of $< K$ results
Champion Lists (7.1.3)	Precompute top r docs per term	Only score docs in champion lists	Top r docs by tf or $tf-idf$	Fast at query time, focuses on strong docs	r fixed $\rightarrow$ may miss good docs; may return $< K$
Static Quality + Ordering (7.1.4)	Use document quality score $g(d)$	Process high-quality docs first	Sort postings by $g(d)$, combine with $tf-idf$	Better ranking (quality + relevance), early good results	Requires extra storage/computation; still may scan many docs
Impact Ordering (7.1.5)	Process high-impact contributions first	Only process top part of postings lists	Sort postings by tf (or impact)	Very efficient; early stopping possible	No common order $\rightarrow$ more complex; inexact results
Cluster Pruning (7.1.6)	Group docs into clusters	Only search one (or few) clusters	Leaders ($\sqrt{N}$), followers	Huge reduction ($N \rightarrow \sqrt{N}$), very fast	May miss relevant docs in other clusters